

RecoverAI — Product Requirements Document

1. Product definition

RecoverAI is a bounded AI-assisted payment-failure recovery decision engine for Razorpay payments.

Primary user: Merchant revenue/payments operations.

Core promise: Given a failed payment, RecoverAI determines whether recovery is worth pursuing, selects the highest-value permitted next action, executes it through controlled tools, verifies the outcome from payment events, and records why every decision occurred.

2. Problem

Merchants can observe failed payment events but still need to decide what to do next. A blanket retry/reminder strategy can waste interventions, retry unsuitable failures, miss recoverable cases, and provide weak evidence about whether recovery actions created incremental value.

RecoverAI focuses on the decision loop rather than message generation.

3. MVP scope

Supported scenarios

- Failed one-time Razorpay payment associated with an Order.
- Failed or incomplete Razorpay Payment Link.

Approved action vocabulary

- `RETRY`
- `CREATE_OR_REUSE_PAYMENT_LINK`
- `OFFER_ALTERNATE_PAYMENT_METHOD`
- `WAIT`
- `ESCALATE`
- `STOP`

The implementation may simulate some communication actions. The product must never imply that simulated communication is a real production channel.

Core states

EVENT_RECEIVED → VERIFIED → DEDUPLICATED → RECOVERY_CASE_CREATED →
DIAGNOSED → PRIORITIZED → ACTION_SELECTED → POLICY_CHECKED →
APPROVED/ESCALATED/STOPPED → ACTION_EXECUTED → OUTCOME_MONITORED →
RECOVERED/FAILED

4. User stories

US1 — Detect

As a merchant operator, I want a failed payment to automatically create a recovery case so I do not manually triage every failure.

Acceptance: A valid failure event creates exactly one case. Invalid signatures are rejected. Duplicate event IDs do not create duplicate cases.

US2 — Diagnose

As an operator, I want the system to explain the likely failure reason and recovery context so I can understand why an action is recommended.

Acceptance: Diagnosis is structured, includes evidence, and is stored with the case.

US3 — Prioritize

As an operator, I want recoverability and expected value considered so high-value/recoverable cases receive appropriate attention.

Acceptance: The case contains a reproducible score and the underlying features used by the scoring layer.

US4 — Decide

As an operator, I want RecoverAI to choose among a small set of approved actions so the next step is contextual rather than blanket.

Acceptance: The model returns typed JSON matching the action schema. Unsupported actions are rejected.

US5 — Control

As a merchant, I want deterministic policy to constrain AI decisions so the model cannot perform prohibited financial actions.

Acceptance: Policy executes after model output and before any tool call. A policy rejection prevents execution and creates an audit event.

US6 — Execute

As an operator, I want approved recovery actions to execute through a controlled adapter so the system demonstrates a real closed loop.

Acceptance: Only approved tools can execute. Tool results are persisted. Test Mode is used for money-related demonstrations.

US7 — Verify

As an operator, I want payment outcomes to close recovery cases so the system measures actual outcomes rather than assuming success.

Acceptance: A verified success event transitions a case to **RECOVERED**. A payment success received after a pending action must prevent further recovery actions.

US8 — Escalate

As an operator, I want uncertain or high-risk cases routed to human review.

Acceptance: Low confidence, high value, policy exceptions, or repeated failures can produce **ESCALATE**; the system never bypasses this gate.

US9 — Audit

As an operator, I want a complete timeline of evidence, decisions, policy checks, tools, and outcomes.

Acceptance: Every state transition and tool attempt has timestamp, actor/source, case ID, decision ID where applicable, and result.

US10 — Benchmark

As a builder, I want to compare RecoverAI with simpler baselines so I can prove or disprove that the AI layer adds value.

Acceptance: A seeded benchmark can run fixed rules, LLM-only, policy-only, and hybrid RecoverAI over the same held-out cases.

5. Success metrics

Primary

- Incremental simulated recovered revenue versus fixed-rule baseline.

Secondary

- Recovery rate.
- Total recovered amount.
- Expected recovered amount.
- Cost-adjusted recovered amount.
- Unnecessary intervention rate.
- Escalation rate.
- Average actions per case.
- Time/actions to recovery.
- Policy violations blocked.

Engineering

- Duplicate-event suppression: 100% in dedicated tests.
- Structured decision validity: $\geq 95\%$.
- Action-after-success errors: $< 5\%$.
- Unnecessary interventions: $< 10\%$.
- Critical policy violations: 0.

6. Eval framework

The evaluation dataset must be generated from a hidden simulator. Each case begins from latent customer/payment state, which determines recoverability and action-specific outcomes. The agent receives only observable features.

The benchmark must be seeded and reproducible.

Compare:

1. Naive fixed action baseline.
2. Fixed rule baseline.
3. LLM-only decision policy.
4. Policy-only system.
5. RecoverAI hybrid.

Use a held-out test set. Do not tune the policy against the final test set.

7. Guardrails specification

Hard rules

- No model-generated direct payment API calls.
- Only enumerated tools can execute.
- No refund execution in the MVP.
- No production money movement.
- No action after a verified successful payment.
- No retry beyond configured retry count.
- No action when customer has explicitly opted out.
- No action above merchant-defined monetary thresholds without escalation.
- No unsupported action type.
- Low-confidence/high-value cases require human review.
- Payment status from verified events is authoritative.

Stop conditions

Stop when payment succeeds, customer declines/opts out, maximum attempts are reached, the case becomes ineligible, or policy blocks further action.

Human escalation

Escalate for ambiguous evidence, high-value cases beyond autonomous thresholds, repeated failures, policy conflicts, tool errors requiring judgment, or any unsupported state.

8. Model strategy

Use a single structured-output LLM for diagnosis, contextual reasoning, and recommendation. Do not build multi-agent orchestration.

Model responsibilities:

- classify/interpret failure context;
- synthesize evidence;
- estimate action suitability;
- return structured recommendation and rationale.

Deterministic services own:

- policy;
- monetary constraints;
- retry counts;
- consent/opt-out;

- tool permissions;
- final state transitions;
- benchmark scoring.

Use a provider abstraction so the model can be changed without redesigning the product. If the model is unavailable, the system should fall back to policy-only behavior rather than fail the financial workflow.

9. Data requirements

Case schema should include at minimum:

- case ID;
- event ID;
- payment/order/link IDs;
- amount/currency;
- failure code and description;
- payment method context;
- customer history summary;
- attempt count;
- elapsed time;
- previous recovery actions;
- consent/opt-out status;
- merchant policy version;
- model decision;
- policy result;
- tool execution result;
- final outcome.

Synthetic data must include recoverable, non-recoverable, ambiguous, duplicate, already-successful, hard-decline, transient-error, and tool-failure scenarios.

Do not use real customer payment data.

10. Responsible AI

- Do not infer sensitive personal attributes.
- Use only operational/payment context necessary for the decision.
- Keep decisions explainable through evidence and structured rationale.
- Log model and policy versions.
- Evaluate error categories separately rather than relying only on aggregate performance.
- Do not present synthetic results as production performance.
- Prefer deterministic control for financially consequential rules.

11. Monitoring plan

Track:

- decision validity;
- policy rejection rate;
- action-after-success errors;
- duplicate-event rate;
- tool failures;
- recovery outcomes;
- escalation rate;
- intervention rate;
- model latency/cost;
- drift in failure-category distribution.

A production implementation would trigger review when safety metrics regress, tool failures rise, failure distributions shift materially, or model output validity falls below the agreed threshold.

12. Definition of done

The MVP is complete only when:

1. A failed payment event can enter the system.
2. Signature validation and deduplication work.
3. A recovery case is created.
4. Context is assembled.
5. AI returns a typed diagnosis/recommendation.
6. Deterministic policy accepts/rejects it.
7. A controlled tool executes an approved action.
8. A payment outcome event verifies success/failure.
9. The case reaches a terminal state.
10. The entire timeline is auditable.
11. The benchmark compares the hybrid policy with baselines.
12. The UI can demonstrate the complete loop.

13. Explicit cut list

If time is constrained, cut in this order:

1. Subscription support.
2. Alternate payment-method sophistication.
3. Advanced dashboard analytics.
4. Visual polish.

5. Anything involving additional communication channels.

Never cut:

- deterministic guardrails;
- webhook verification/deduplication;
- auditability;
- benchmark;
- end-to-end closed loop.